DSA - Microsoft

Two Sum

```
class Solution {
public:
    vector<int> twoSum(vector<int>& nums, int target) {
        unordered_map<int,int> mp;

        for (int i = 0; i<nums.size(); i++){
            int complement = target - nums[i];

            if (mp.count(complement))
                return {mp[complement], i};

            mp[nums[i]] = i;
        }

        return {};
    }
};
```

Longest Substring without repeating characters

```
class Solution {
public:
    int lengthOfLongestSubstring(string s) {
        unordered_map<char,int> mp;
        int left = 0;
        int maxLen = 0;

        for (int right = 0; right<s.size(); right++){
            if (mp.find(s[right]) != mp.end())
            left = max(left, mp[s[right]]+1);

            mp[s[right]] = right;
            maxLen = max(maxLen, right - left +1);
        }

        return maxLen;
    }
};
```

Merge Sorted Array

```
class Solution {
public:
```

```
    void merge(vector<int>& nums1, int m, vector<int>&
nums2, int n) {
        int i = m-1;
        int j = n-1;
        int k = m+n-1;

        while(j>=0){
            if (i>=0 && nums1[i] > nums2[j]){
                nums1[k--] = nums1[i--];
            }else{
                nums1[k--] = nums2[j--];
            }
        }
    }
};
```

Maximum Subarray

```
class Solution {
public:
    int maxSubArray(vector<int>& nums) {
        int n = nums.size();
        vector<int> dp(n);//create dp array

        dp[0]=nums[0];
        int ans=dp[0];//create ans integer

        for (int i=1;i<n;i++){
            dp[i] = max(nums[i],dp[i-1]+nums[i]);
            //max sum subaarray
            ans = max(ans,dp[i]);
        }

        return ans;
    }
};
```

Climbing stairs

```
class Solution {
public:
    int climbStairs(int n) {
        if (n<=2)
            return n;

        vector<int> dp(n+1);
```

```
        dp[1] = 1;
        dp[2] = 2;

        for (int i=3; i<=n; i++){
            dp[i] = dp[i-1] + dp[i-2];
        }

        return dp[n];
    }
};
```

Number of islands

```
class Solution {
public:
    void dfs(int i, int j, vector<vector<char>> &grid){
        if (i<0 || i>=grid.size() || j<0 ||
j>=grid[0].size() || grid[i][j] == '0')
        return;

        grid[i][j] = '0';//cell is visited

        dfs(i+1,j,grid);
        dfs(i-1,j,grid);
        dfs(i,j+1,grid);
        dfs(i,j-1,grid);
    }

    int numIslands(vector<vector<char>>& grid) {
        int m = grid.size();
        int n = grid[0].size();
        int count = 0;

        for (int i=0; i<m; i++){
            for (int j=0; j<n; j++){
                if (grid[i][j] == '1'){
                count++;
                dfs(i,j,grid);
                }
            }
        }

        return count;
    }
};
```

Sub-Array Sum equals k

```
class Solution {
public:
    int subarraySum(vector<int>& nums, int k) {
        unordered_map<int,int> mp;
        mp[0] = 1;

        int prefixSum = 0;
        int count = 0;

        for (int num:nums){
            prefixSum += num;
            if (mp.count(prefixSum-k))
            count += mp[prefixSum-k];

            mp[prefixSum]++;
        }

        return count;
    }
};
```

Best time to buy and sell stock

```
class Solution {
public:
    int maxProfit(vector<int>& prices) {
        int minPrice = INT_MAX;
        int maxProfit = 0;

        for (int price:prices){
            minPrice = min(minPrice, price);
            maxProfit = max(maxProfit, price - minPrice);
        }

        return maxProfit;
    }
};
```

Valid Parenthesis

```
class Solution {
public:
    bool isValid(string s) {
        stack <char> st;
```

```
        for (char ch:s){
            if (ch=='(' || ch=='{' || ch=='[')
                st.push(ch);

            else{
                if (st.empty())
                    return false;

                if ((ch==')'&&st.top()!='(')||
(ch=='}'&&st.top()!='{')||(ch==']'&&st.top()!='['))
                    return false;

                st.pop();
            }
        }
        return st.empty();
    }
};
```

Merge two sorted lists

```
/**
 * Definition for singly-linked list.
 * struct ListNode {
 *     int val;
 *     ListNode *next;
 *     ListNode() : val(0), next(nullptr) {}
 *     ListNode(int x) : val(x), next(nullptr) {}
 *     ListNode(int x, ListNode *next) : val(x), next(next)
{}
 * };
 */
class Solution {
public:
    ListNode* mergeTwoLists(ListNode* list1, ListNode*
list2) {
        if (list1==NULL)
            return list2;

        if(list2==NULL)
            return list1;

        if (list1->val<list2->val){
            list1->next = mergeTwoLists(list1->next, list2);
            return list1;
        }else{
```

```
            list2->next = mergeTwoLists(list1, list2->next);
            return list2;
        }
    }
};
```

Search in a rotated sorted array

```
class Solution {
public:
    int findPivot(vector<int>& nums, int n){
        int l = 0, r = n-1;
        while(l<r){
            int mid = l+(r-l)/2;

            if (nums[mid]>nums[r]){
                l =mid+1;
            }else{
                r=mid;
            }
        }
        return r;
    }

    int binarySearch(int l, int r, vector<int>& nums, int
target){

        while(l<=r){
            int mid = l+(r-l)/2;

            if (nums[mid]==target)
                return mid;

            else if (nums[mid]<target)
                l = mid+1;
            else
                r = mid-1;
        }
        return -1;
    }

    int search(vector<int>& nums, int target) {
        int n = nums.size();
        int pivotIdx = findPivot(nums,n);//idx of min
element

        int idx = binarySearch(0,pivotIdx-1,nums,target);//
```

```
left half

        if (idx!=-1)
            return idx;

        idx = binarySearch(pivotIdx, n-1, nums, target);//
right half

        return idx;
    }
};
```

Reverse Linked Lists

```
class Solution {
public:
   ListNode* reverseList(ListNode* head) {
      ListNode* prev = nullptr;

      while(head){
         ListNode* next = head->next;
         head->next = prev;
         prev = head;
         head = next;
      }

      return prev;
   }
};
```

Spiral Matrix

```
class Solution {
public:
   vector<int> spiralOrder(vector<vector<int>>& matrix) {
      vector<int> ans;

      int top = 0;
      int bottom = matrix.size()-1;
      int left = 0;
      int right = matrix[0].size()-1;

      while(top<=bottom && left<=right){
         //left to right
         for (int j=left; j<=right; j++){
            ans.push_back(matrix[top][j]);
         }
```

```
            top++;

            //top to bottom
            for (int i=top; i<=bottom; i++){
                ans.push_back(matrix[i][right]);
            }
            right--;

            //right to left
            if (top<=bottom){
                for(int j=right; j>=left; j--){
                    ans.push_back(matrix[bottom][j]);
                }
                bottom--;
            }

            //bottom to top
            if (left<=right){
                for (int i=bottom; i>=top; i--){
                    ans.push_back(matrix[i][left]);
                }
                left++;
            }
        }

        return ans;
    }
};
```

Group Anagrams

```
class Solution {
public:
    vector<vector<string>> groupAnagrams(vector<string>& strs) {
        unordered_map<string, vector<string>> mp;

        for (string str:strs){
            string key = str;
            sort(key.begin(),key.end());

            mp[key].push_back(str);
        }

        vector<vector<string>> ans;
        for (auto &it:mp){
            ans.push_back(it.second);
        }
```

```
        return ans;
    }
};
```

Container with most water

```
class Solution {
public:
    int maxArea(vector<int>& height) {
        int left = 0;
        int right = height.size()-1;
        int maxWater = 0;

        while(left<right){
            int minHeight = min(height[left], height[right]);
            int width = right - left;
            maxWater = max(maxWater, minHeight*width);

            if (height[left]<height[right]){
                left++;
            }else{
                right--;
            }
        }
        return maxWater;
    }
};
```

Product of Array except self

```
class Solution {
public:
    vector<int> productExceptSelf(vector<int>& nums) {
        int n = nums.size();
        vector<int> ans(n,1);
        int prefix = 1;
        int suffix = 1;

        for (int i=0;i<n;i++){
            ans[i] = prefix;
            prefix*=nums[i];
        }

        for (int i=n-1; i>=0; i--){
            ans[i]*=suffix;
            suffix*=nums[i];
```

```
        }

        return ans;
    }
};
```

Word search

```
class Solution {
public:
    bool dfs(vector<vector<char>>& board, int i, int j, string word, int idx){
        if (idx == word.size())
            return true;

        if (i<0 || i>=board.size() || j<0 || j>=board[0].size() || board[i][j]!=word[idx])
            return false;

        char temp = board[i][j];//mark cell as visited
        board[i][j] = '#';

        bool found =
            dfs(board, i+1, j, word, idx+1)||
            dfs(board, i-1, j, word, idx+1)||
            dfs(board, i, j+1, word, idx+1)||
            dfs(board, i, j-1, word, idx+1);

        board[i][j] = temp;//backtrack

        return found;
    }

    bool exist(vector<vector<char>>& board, string word) {
        int m = board.size();
        int n = board[0].size();

        for (int i=0; i<m; i++){
            for (int j=0; j<n; j++){
                if (board[i][j]==word[0]){
                    if (dfs(board,i,j,word,0))
                        return true;
                }
            }
        }
        return false;
    }
};
```